Uniwersytet w Siedlcach
Wydział Nauk Ścisłych

Instytut Informatyki

Programowanie Zaawansowane

Zadanie projektowe nr.1 – The Binding Of Isaac

Prowadzący laboratorium:
Dr. Waldemar Bartyna

Opracowanie:

Dawid Dziedzic

Rok III, Inż., Gr.1

Siedlce, 2024

Wprowadzenie

Zadanie polegało na możliwie jak najwierniejszym odwzorowaniu jednej z topowych gier typu roguelike - The Binding Of Isaac, w oparciu o poznane technologie i funkcje dostępne w języku C#.

Symulator umożliwia graczowi zagłębienie się w proceduralnie generowane środowisko, pełne losowych pięter, pokoi, przeciwników i przedmiotów, które definiują unikalny charakter każdej rozgrywki gry The Binding Of Isaac.

Link do repozytorium

https://github.com/Daion5/TheBindingOfIsaac.git

Spis funkcjonalności

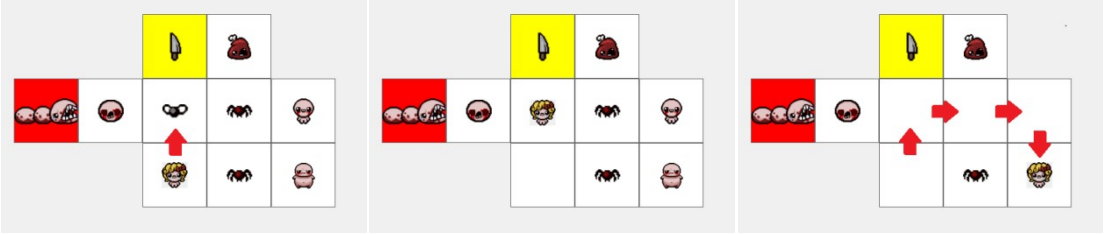
- Wybór postaci

Po uruchomieniu aplikacji, użytkownik może wybrać 1 z 3 dostępnych postaci (Isaac, Judas, Magdalene). Wybór postaci determinuje unikalne statystyki oraz wygląd gracza podczas rozgrywki.

Rysunek 1 Menu z wyborem postaci.

- Przemieszczanie się pomiędzy pokojami

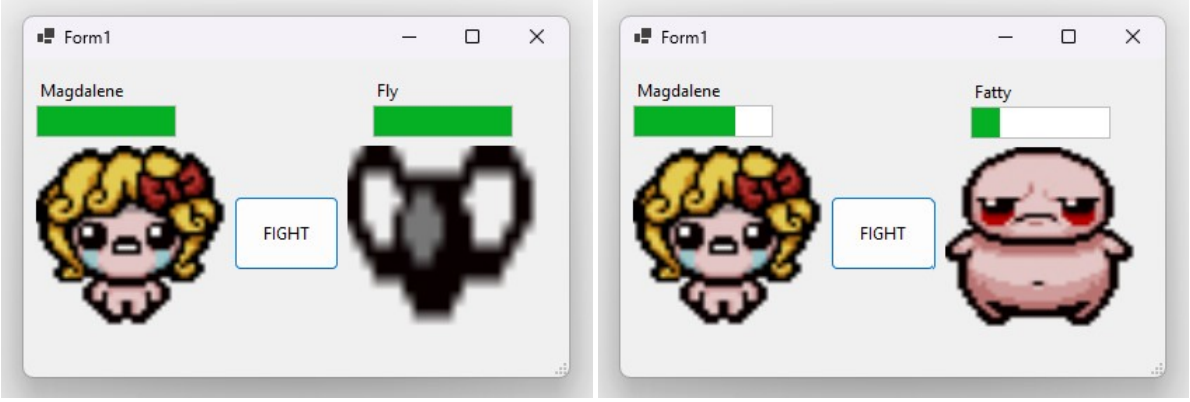
Po wybraniu postaci z menu, gracz może swobodnie przemieszczać się pomiędzy pokojami za pomocą klawiszy strzałek. Kliknięcie strzałki we wskazanym kierunku spowoduje przeniesienie się gracza do wybranego pokoju.



Rysunek 2, 3, 4 Prezentacja poruszania się pomiędzy pokojami.

- Walka z przeciwnikami

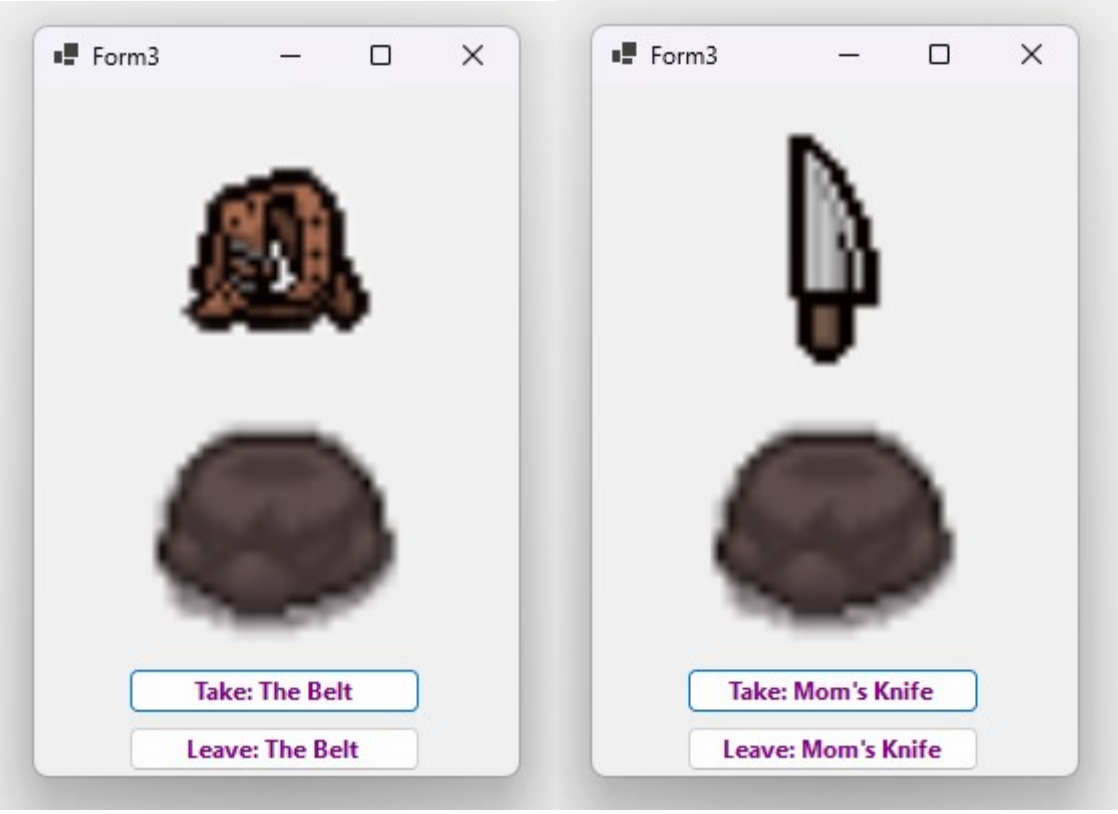
W trakcie eksploracji piętra gracz zobligowany jest do pokonywania przeciwników stających mu na drodze do wygranej. Każdorazowe przejście postaci do nowego „zwykłego" pokoju, skutkuje wywołaniem okna walki z potworem w nim się znajdującym. Gracz aby przejść dalej musi stoczyć i wygrać pojedynek. Symulacja walki odbywa się za pomocą naciskania przycisku „Fight" gdzie walka toczy się na regułach turowych.



Rysunek 5, 6 Prezentacja okna walki.

- Zdobywanie przedmiotów

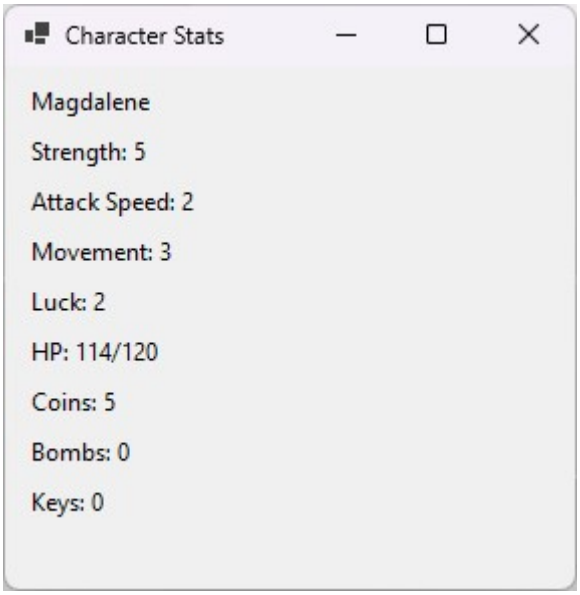
W specjalnych pokojach oznaczonych na mapie za pomocą koloru żółtego, gracz może znaleźć różne przedmioty wzmacniające statystyki gracza. Zdobywanie przedmiotów w odpowiednim momencie znacząco ułatwia rozgrywkę.



Rysunek 7, 8 Prezentacja okna z przedmiotem.

- Wyświetlanie statystyk postaci (Tab)

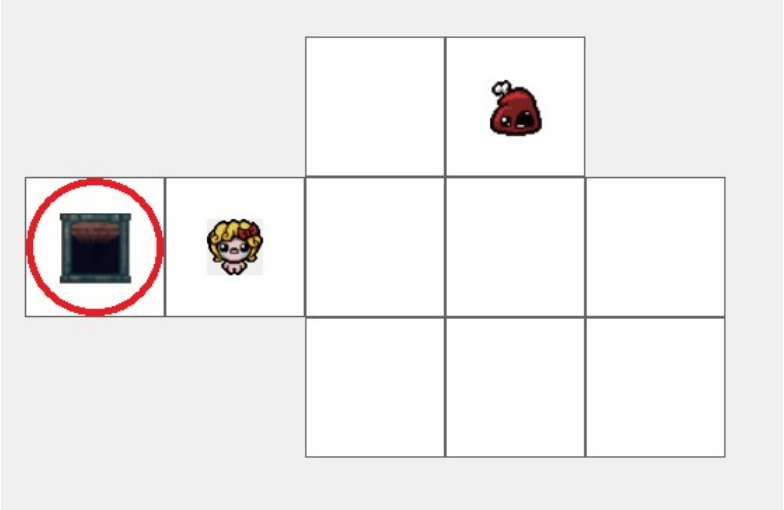
W dowolnym momencie gry, gracz może sprawdzić statystyki swojej postaci wciskając klawisz Tab. Mechanizm ten w przejrzysty sposób umożliwia graczowi zapoznanie się z podnoszonymi przedmiotami i bonusami które oferują, ułatwia również monitorowanie obecnie posiadanej ilości zdrowia postaci.



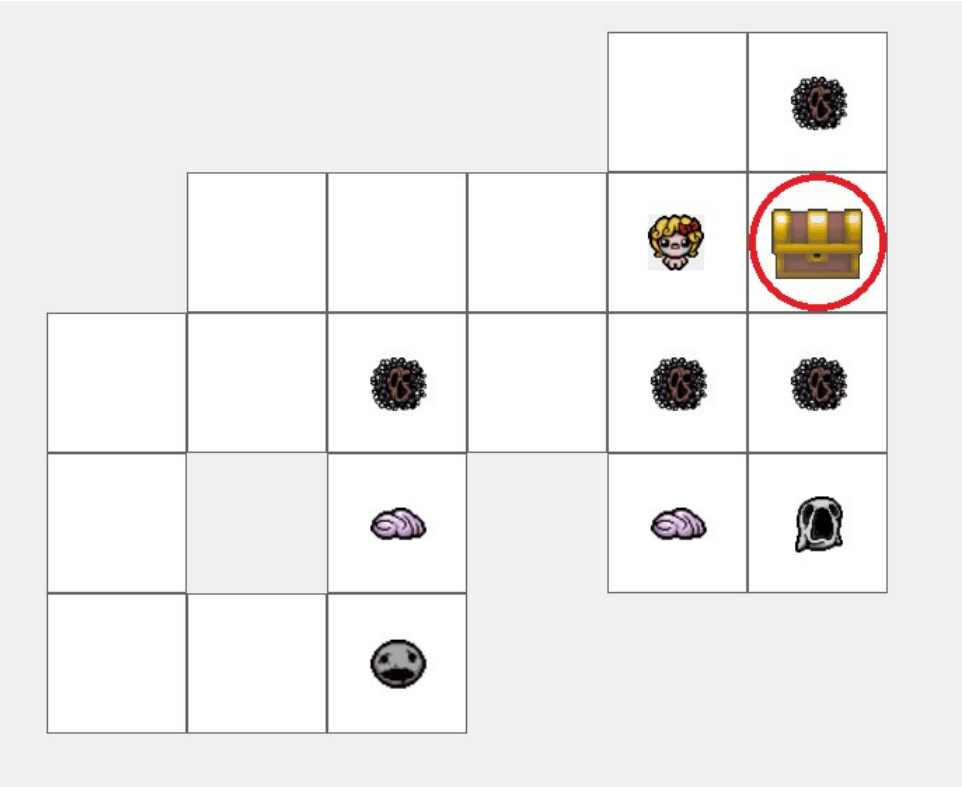
Rysunek 9 Prezentacja okna statystyk.

- Wchodzenie w interakcję z dynamicznie generowanym otoczeniem (Enter)

Po pokonaniu bossa aktualnego piętra (czerwony pokój), przed graczem zostanie wygenerowany specjalny obiekt który będzie wymagał interakcji z użytkownikiem. Aby przejść na dalsze piętra i móc ukończyć grę gracz musi znajdować się w pokoju z wygenerowanym obiektem i nacisnąć klawisz „Enter" który spowoduje przeniesienie gracza na dalsze piętra gry, lub ukończenie gry w zależności od wygenerowanego obiektu – dopiero na ostatnim 3 piętrze zabicie bossa generuje obiekt umożliwiający ukończenie gry.



Rysunek 10 Wygenerowany obiekt „trapdoor".



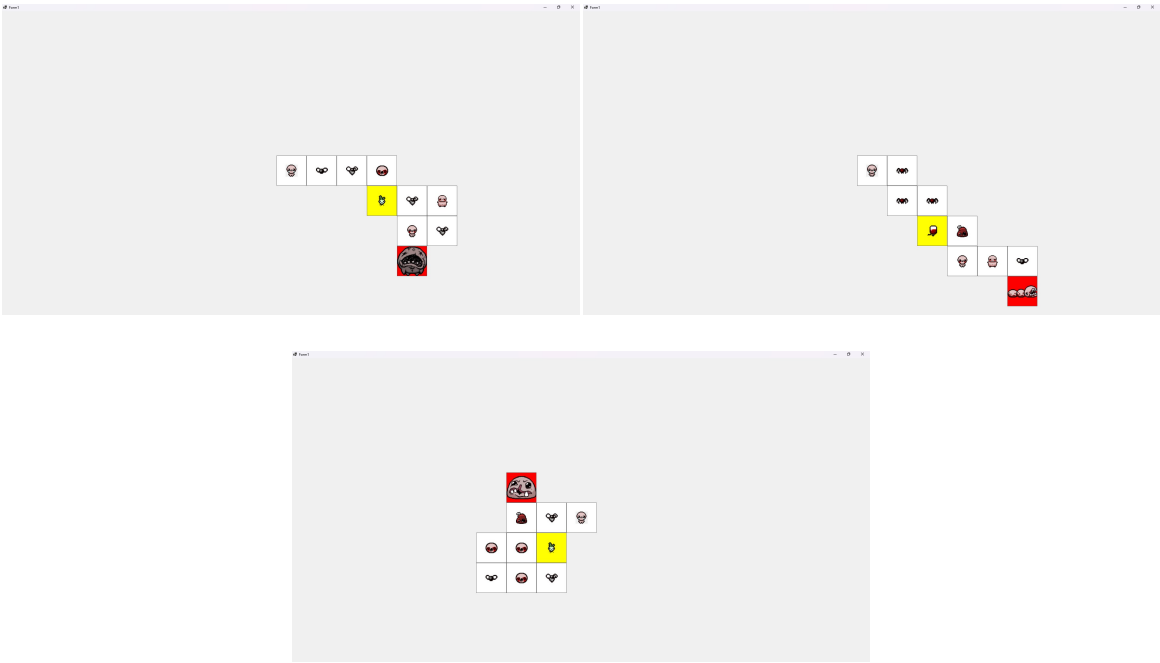
Rysunek 11 Wygenerowany obiekt „ending_chest".

Najciekawsze funkcjonalności

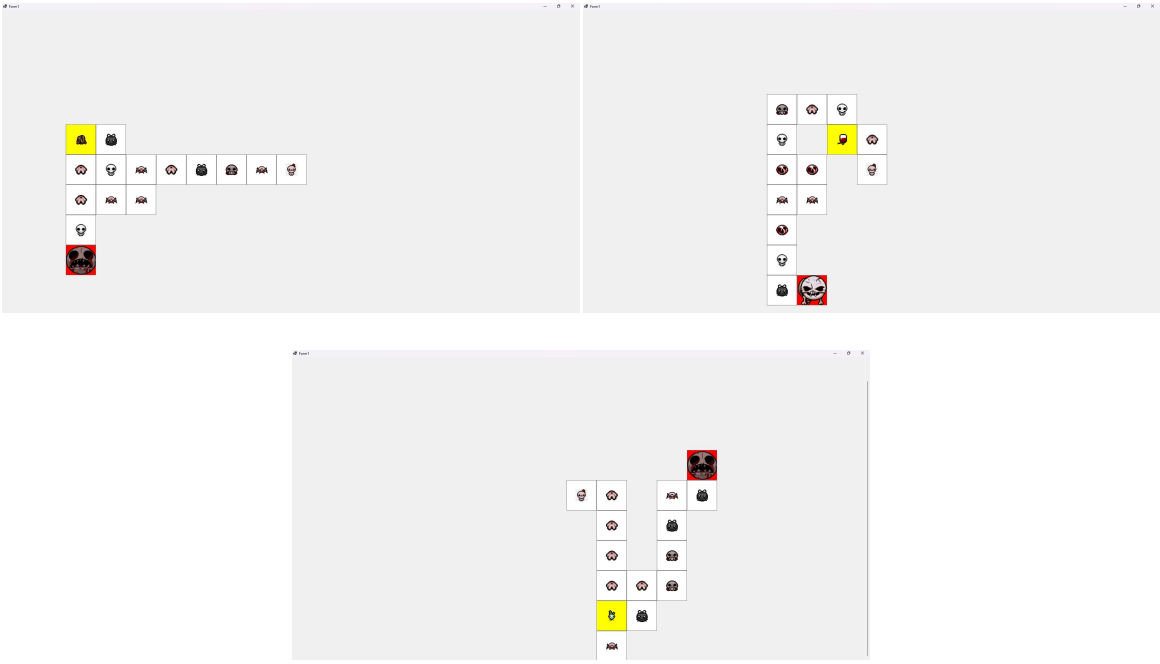
- Mechanizm generowania losowych pięter

Jest to autorski i jeden z kluczowych elementów rozgrywki, który sprawia, że każda gra jest unikatowa i niepowtarzalna. Każde piętro generowane jest proceduralnie, co oznacza, że układ pokoi i rozmieszczenie przedmiotów, przeciwników oraz bossów różni się w każdym podejściu.

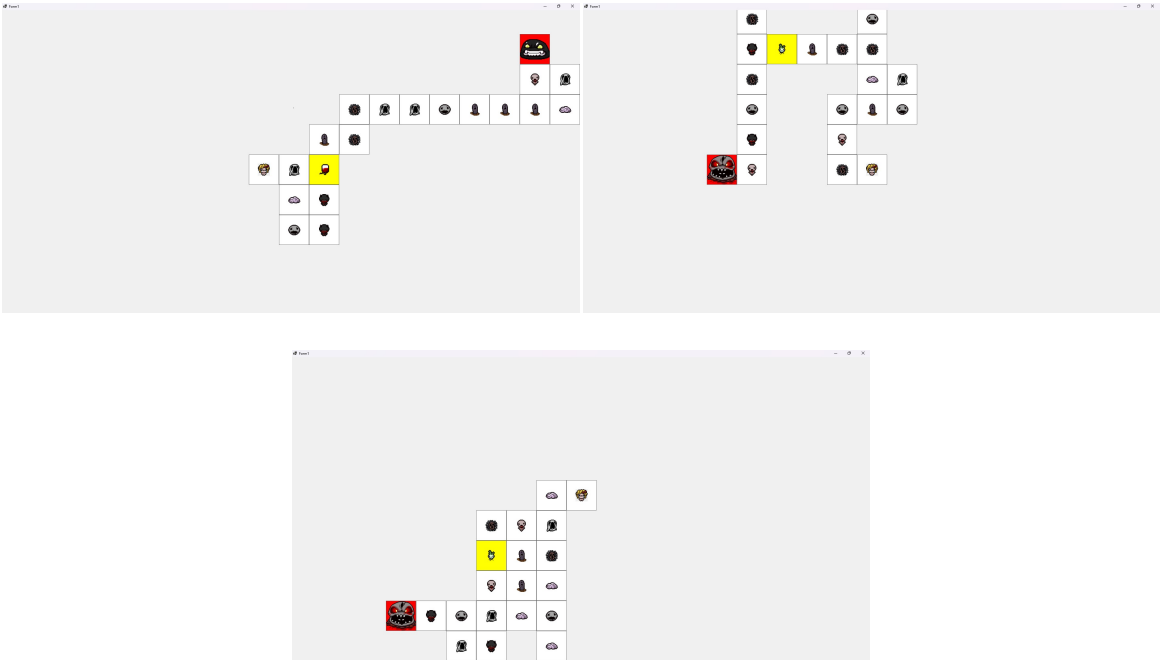
Szczegóły techniczne:
Algorytm zaczyna od wygenerowania pokoju startowego w centrum ekranu. Jest to startowy pokój gracza nie zawierający niczego poza samym graczem. Na podstawie zdefiniowanych reguł, algorytm generuje pokoje rekurencyjnie, aż do osiągnięcia określonej liczby pokoi na piętrze (1 piętro = 10 pokoi, 2 piętro = 15 pokoi, 3 piętro = 20 pokoi). Algorytm tasuje listę krawędzi (Right, Bottom, Left, Top) a następnie wybiera jedną nich. Sprawdza czy po wybranej stronie (wylosowana krawędź) istnieje już pokój (metoda hasNeighbor()). Jeśli nie, sprawdza, czy nowy panel zmieści się w oknie formularza Windows Forms (metoda IsInsideWindow()). Jeśli wszystkie warunki są spełnione (pokój mieści się w formularzu i nie ma sąsiada), generuje nowy pokój w odpowiednim miejscu. Proces powtarza się, aż do uzyskania odpowiedniej ilości pokoi na piętrze. Każde z wygenerowanych pokoi otrzymuje Tag „Monster” co umożliwia wygenerowanie w tym pokoju przeciwnika. Wyjątkiem jest ostatni wygenerowany pokój z tagiem „Boss”. Dodatkowo losowo wybrany zwykły pokój który miałby być oznaczony tagiem „Monster”, zmienia się na pokój z przedmiotem oznaczony Tagiem „Item”.



Rysunek 12, 13, 14 Wygenerowane piętra – poziom I”.



Rysunek 15, 16, 17 Wygenerowane piętra – poziom II”.



Rysunek 18, 19, 20 Wygenerowane piętra – poziom III”.

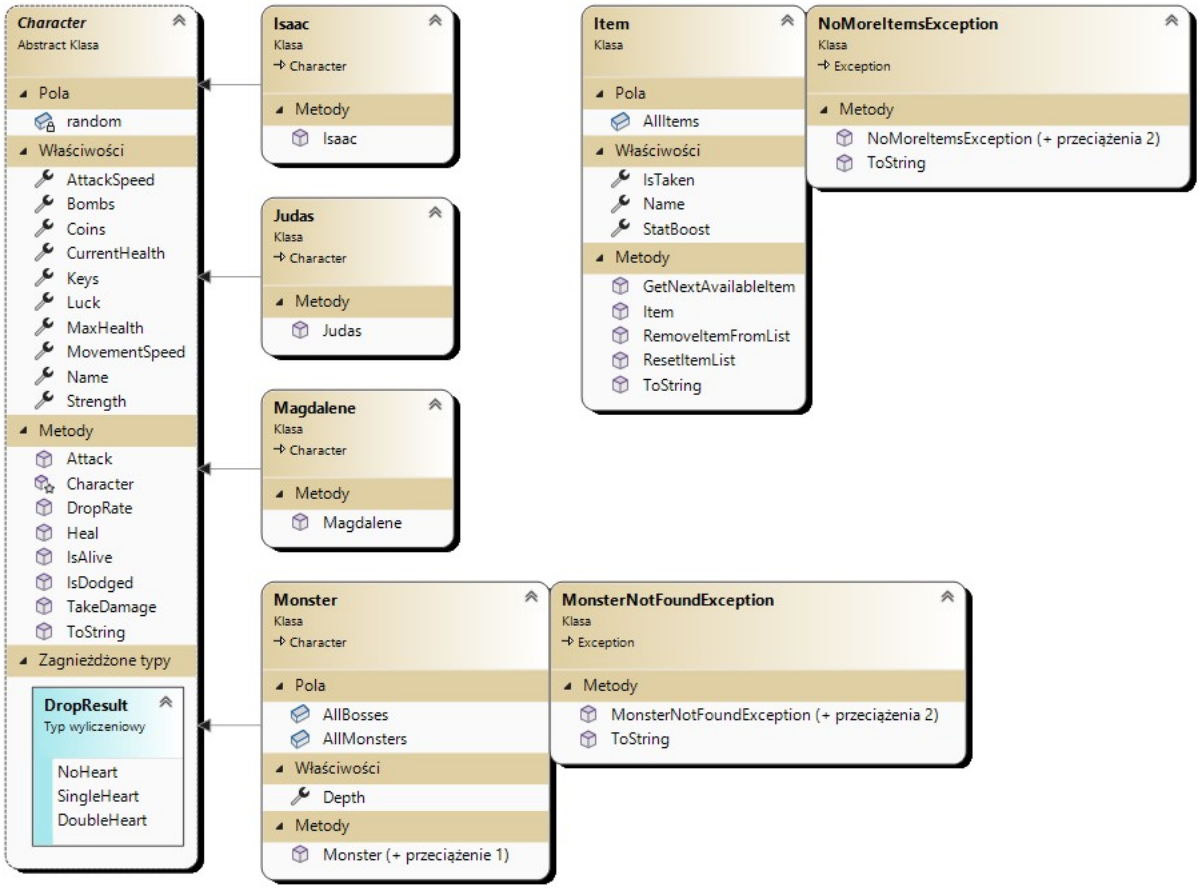
- System zapisu wyników rozgrywek postaci

System śledzi poczynania gracza w każdej rozgrywce i zapisuje wynik gry wygranej bądź przegranej, aktualizując w ten sposób wynik „pasma zwycięstw” znajdującego się w lewym dolnym rogu naszego menu wyboru postaci. W ten sposób gdy gracz wygra grę całkowicie, system zwiększa nasze pasmo o 1, a gdy przegra, zmniejszy pasmo o 1. Dodatkowo najechanie kursorem myszy na napis „win_streak” powoduje wyświetlenie nam okna z informacją o najwyższym i najniższym posiadanym paśmie zwycięstw lub porażek. Cały mechanizm opiera się na serializacji danych do pliku userData.xml.

Rysunek 21 Prezentacja dodatkowej funkcjonalności win-streak’u.

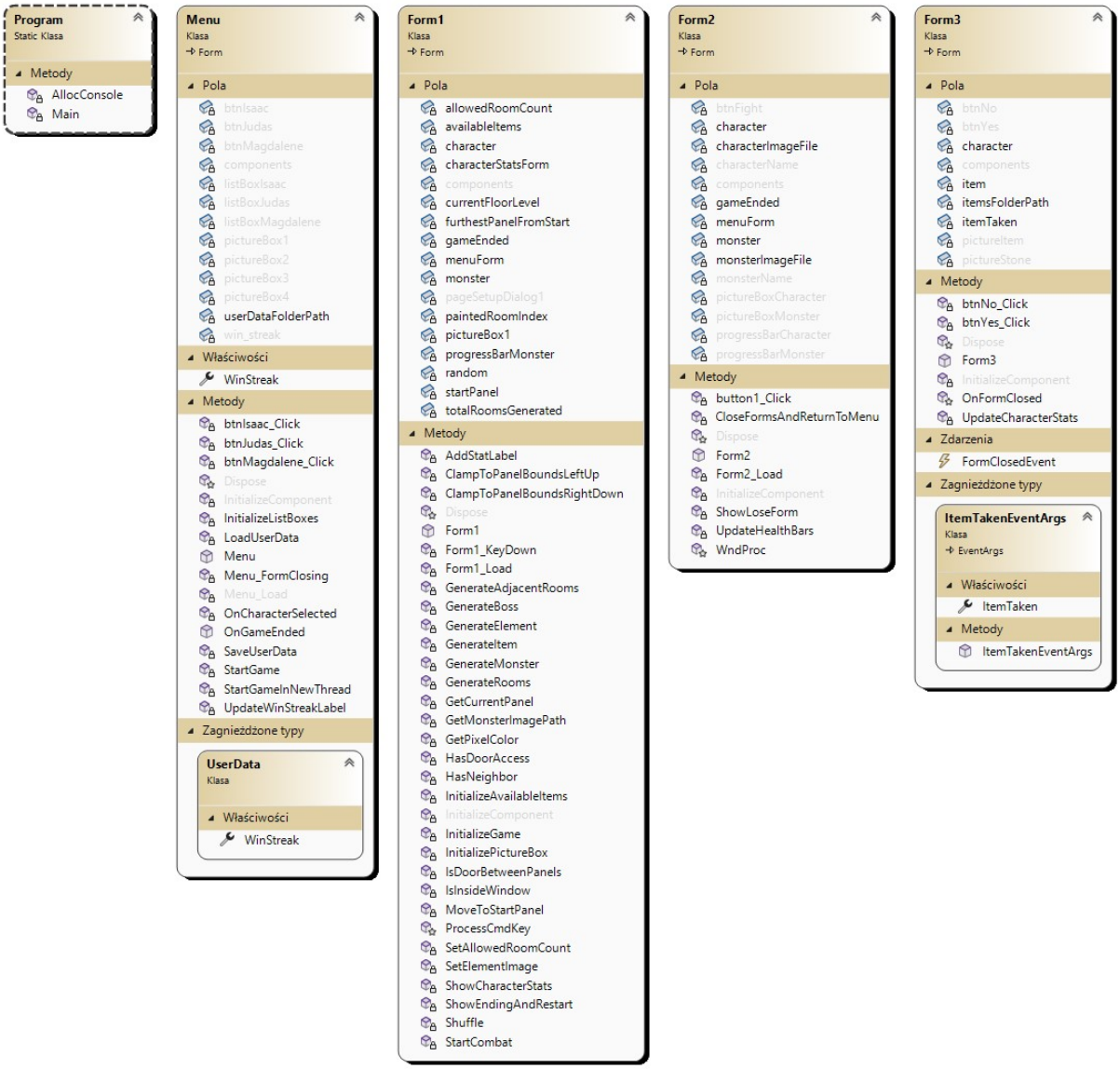
Diagramy klas

Library:



Rysunek 22 Diagram klas „Library”.

TheBindingOfIsaac:



Rysunek 23 Diagram klas „TheBindingOfIsaac”.